FORMATION « PYTHON EN 24 HEURES » | SÉANCE 02

Répéter sans se répéter : boucles et algorithmes

Chapitres 3 et 4 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Choisir entre la boucle TANT QUE et la boucle POUR selon la situation
- Utiliser un compteur et un accumulateur pour compter et additionner
- Écrire des boucles imbriquées pour traiter des tableaux à deux dimensions
- Implémenter la recherche du minimum, du maximum et le tri par sélection
- Identifier les deux pièges majeurs : la boucle infinie et le décalage de un

Rappel de la séance 01

Avant de répéter, revoyons les briques que vous maîtrisez déjà : elles seront utilisées à l'intérieur de chaque boucle.

- Variables et affectation : une boîte étiquetée qui garde une valeur
- ENTRER et AFFICHER : dialoguer avec l'utilisateur
- Structures SI, SINON, SINON SI : choisir un chemin
- Opérateurs de test et connecteurs ET, OU, NON
- Méthode : analyser, décrire en pseudo-code, tester à la main

Pourquoi répéter ?

- Afficher la table de 7 : dix instructions identiques ? Inacceptable
- Additionner 200 notes à la main : source d'erreurs garantie
- La boucle répète un bloc d'instructions **sans recopier le code**
- Deux outils selon la situation : **TANT QUE** (tant qu'il faut) et **POUR** (un nombre connu de fois)

La boucle TANT QUE

```
ALGO CompteARebours
compteur reçoit 5
TANT QUE compteur > 0 FAIRE
    AFFICHER compteur
    compteur reçoit compteur moins 1
FIN TANT QUE
AFFICHER "Décollage !"
```

Trois ingrédients : initialisation avant la boucle, condition de poursuite, mise à jour à l'intérieur.

La boucle POUR

```
ALGO TableDe7
POUR k ALLANT DE 1 A 10 FAIRE
    AFFICHER k, " fois 7 = ", k fois 7
FIN POUR
```

Quand le nombre de répétitions est connu d'avance, POUR est plus clair que TANT QUE.

Choisir la bonne boucle

POUR

- Nombre de tours **connu** à l'avance
- Parcours d'une suite ou d'un tableau
- Le compteur est géré automatiquement

TANT QUE

- Nombre de tours **inconnu** à l'avance
- Arrêt dicté par une condition
- Attention : la condition doit finir par devenir fausse

Compteur et accumulateur

```
ALGO MoyenneDe10Notes
somme reçoit 0
POUR i ALLANT DE 1 A 10 FAIRE
    ENTRER note
    somme reçoit somme + note
FIN POUR
moyenne reçoit somme / 10
AFFICHER "Moyenne : ", moyenne
```

L'accumulateur 'somme' démarre à 0 et grossit d'un terme à chaque tour : schéma à retenir toute l'année.

Minimum et maximum d'une série

- Idée : garder le champion courant dans une variable
- Au premier élément, le champion est ce premier élément
- Chaque nouvel élément remplace le champion s'il est plus petit (ou plus grand)
- Ne jamais initialiser à 0 : une série de notes négatives donnerait un faux maximum

```
ALGO MaximumDeLaSerie
ENTRER n          # combien de valeurs
ENTRER premier
maxi reçoit premier
POUR i DE 2 A n FAIRE
    ENTRER valeur
    SI valeur > maxi ALORS
        maxi reçoit valeur
    FIN SI
FIN POUR
AFFICHER "Maximum : ", maxi
```

Boucles imbriquées

```
ALGO GrilleDeProduits
POUR ligne ALLANT DE 1 A 3 FAIRE
    POUR colonne ALLANT DE 1 A 4 FAIRE
        AFFICHER ligne, colonne
    FIN POUR
FIN POUR
```

Pour chaque tour de la boucle externe, la boucle interne exécute sa propre boucle complète : 3 fois 4 tours au total.

Recherche dans un tableau

- Problème : la valeur v figure t elle dans la liste ?
- Parcourir les éléments un à un et **sortir dès la première rencontre**
- Si la boucle se termine sans trouver, la réponse est : absent
- Complexité : au pire n comparaisons pour n éléments

Recherche avec témoin

```
ALGO RechercheDeV
trouve reçoit faux
POUR i ALLANT DE 1 A n FAIRE
    SI tableau[i] = v ALORS
        trouve reçoit vrai
    FIN SI
FIN POUR
SI trouve = vrai ALORS
    AFFICHER "Présent"
SINON
    AFFICHER "Absent"
FIN SI
```

Le témoin ‘trouve‘ évite d’afficher plusieurs fois le même résultat et reste simple à traduire.

Tri par sélection : l'idée

```
ALGO TriParSelection
POUR i ALLANT DE 1 A n moins 1 FAIRE
    posMini reçoit i
    POUR j ALLANT DE i + 1 A n FAIRE
        SI tableau[j] < tableau[posMini] ALORS
            posMini reçoit j
        FIN SI
    FIN POUR
    échanger tableau[i] et tableau[posMini]
FIN POUR
```

On cherche le plus petit élément restant et on le place à sa position définitive, tour après tour.

Attention

Les deux pièges classiques

La boucle infinie : si la variable de la condition n'est jamais mise à jour, le TANT QUE ne s'arrête jamais. Le décalage de un : de 1 à n fait n tours, de 0 à n en fait n + 1 ; décidez toujours explicitement des bornes. Dans les deux cas, exécutez la boucle à la main sur un petit exemple avec trois tours.

À retenir

L'essentiel de la séance 02

POUR quand on compte, TANT QUE quand on attend un événement. Le schéma accumulateur (somme qui démarre à 0) et le schéma champion courant (maximum initialisé au premier élément) reviennent dans tous les algorithmes. Le tri par sélection combine boucles imbriquées et échange : apprenez à le dérouler à la main, c'est le meilleur moyen de comprendre toutes les boucles.

siques

Cinq exercices pour muscler vos boucles : comptage, accumulation, test de primalité et un tri complet. Écrivez d'abord la logique en pseudo-code pour les exercices 3 à 5, puis codez.

- **Exercice 1 (Basique)** : Table de multiplication
- **Exercice 2 (Basique)** : Somme et moyenne d'une série
- **Exercice 3 (Intermédiaire)** : Factorielle
- **Exercice 4 (Intermédiaire)** : Nombre premier
- **Exercice 5 (Défi)** : Tri par sélection pas à pas

Ouvrez le notebook « Seance_02_boucles_et_algorithmes.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 03 : premiers pas concrets en Python. Nous installerons nos habitudes d'interpréteur, découvrirons les types int, float, str et bool, les conversions et les f-strings. Refaites la table de 7 avec un print par ligne pour vous échauffer.